PS1 Predictive Analytics

Barni

Aim of this assignment is to learn:

- how to execute commands in R
- how to get started download R, RStudio and open up RStudio.(If you haven't used Rstudio before, search Youtube for "rstudio tutorial for beginners". There are also various other tutorials online.)
- how to do descriptive statistics
- how to write a simple function
- how to solve basic problems

1 Basics

1.1

Use the caschool.csv dataset. We're going to use the readr package which makes it very convenient for reading data. For reading in excel files (we'll primarily use csv files in this course) see the readxl package. The caschool.csv file has information on California schools.

Import the dataset

```
library(readr) #load the readr package
caschool <- read_csv("caschool.csv")
```

Print the first few rows (However, simply writing the name of the dataset will allow you to print the dataset).

```
print(caschool)
```

Get the names of the data. For now, don't worry about what the variables mean.

```
names(caschool)
```

1.2

Get printed summary statistics of the data

```
summary(caschool)
```

1.3 Run a Regression

Here, we'll use the lm() function to run a regression and look at a summary of the output. Note how the <- operator assigns output to a variable. testscr is the student standardized test scores, meal_pct is the percentage of students receiving a government subsidized lunch, and comp_stu is the number of computers per student.

Run a simple linear regression testscr on meal_pct

```
#Simple linear regression
model.1 <- lm(testscr ~ meal_pct, caschool)
summary(model.1)
```

Run a multivariate regression testscr on meal_pct & comp_stu

```
#Multivariate regression
model.2 <- lm(testscr ~ meal_pct + comp_stu, data = caschool)
summary(model.2)
```

1.4 Draw a scatterplot

Here, we'll use the ggplot2 package that is automatically loaded with the tidyverse package

```
library(tidyverse,warn.conflicts=FALSE) #
p <- ggplot(caschool, aes(x = meal_pct, y = testscr)) + #The basis for the plot
geom_point() + #Add points for a scatterplot
geom_smooth(method = lm) + #Add a regression line
labs(x = "Percentage of Students Receiving a Free Lunch",
y = "Test Score",
title = "California Schools")
print(p)
```

The fun thing about R is that there are many different ways to get stuff done. An alternative way to produce the plot without using the ggplot2 package is to use the plot function and add a fitted regression line to it.

```
plot(caschool$meal_pct, y = caschool$testscr);abline(lm(testscr~meal_pct
                                                        , data = caschool), col = "red")
```

1.5 Getting Help

To get help for a specific function, use **?***function*, this searches only within loaded packages. Use **??***function* if you are not sure if a function is in the loaded packages. You can search for a more general topic using the help.search() function. For further guidance, see the tips posted in the previous exercise.

```
?mean
```

1.6 The pipe operator

The R pipe operator %>% automatically pipes the output of one expression to the next expression. This makes chaining R expressions together easy and convenient. The best way to see this is through an example. Please generate random numbers from the normal distribution, then take their mean, and print the output.

```r
x <- rnorm(n = 1000, mean = 0, sd = 2)
norm.mean <- mean(x)
print(norm.mean)
```

Do the same thing with the pipe operator. Again, the pipe operator is going to "pipe" the output from the last expression to the next expression. The `.` is holding the output from the previous expression.

```r
rnorm(n = 1000, mean = 0, sd = 2) %>%
mean(.) %>% #Take the mean
print(.) #Print
```

To see the power of pipes, use dplyr::filter to regress testscr on meal_pct only for the county of Los Angeles.

```r
caschool %>%
filter(county == "Los Angeles") %>% #filter data
lm(testscr ~ meal_pct, data = .) %>% #run regression
summary %>% #take summary of regression output
print #print regress output
```

2 Basic Problems

Based on material from chapter 1&2 in AppliedEconometricswithR

2.1

Create a vector with values that range from 1 to 10, get the length of the vector, square each element using sapply(), vector multiply x'x and print the first and last element of x.

```r
#Solution
#Vector from 1 to 10 -- a few different ways
x <- 1:10
x <- seq(from=1,to=10,by=1)
x <- c(1,2,3,4,5,6,7,8,9,10)
#Length of x
length(x)
#Square each element using sapply
sapply(x,function(number) number^2)
#also can just use
x^2
#vector multiply x'x
t(x)%*%x
#print the first and last elements
x[1]
x[length(x)]
x[10] ## length(x) == 10
```

2.2

Load the "bdims" dataset into R. Filter and rename the variables "hgt" and "wgt".

(a) View the data, look at the head() function, look at the tail using the tail() function, and view the class of dataset.

```
#Solution
#Load in some preset data. The height and weight for
# 507 individuals
install.packages("openintro")
library(openintro)
data(bdims)
bdims = subset(bdims, select = c(wgt,hgt))
bdims <- bdims %>%
  rename(
    weight = wgt,
    height = hgt
  )
#view the data
bdims
#view the first six observations
head(bdims)
#view the last six observations
tail(bdims)
#view the class of the dataset
class(bdims)
```

(b)

Select the first and second columns by the column name and the column number. Select first and last rows by row number, and select the tenth row for the second column.

```
#Solution
#Select the first column in the "dataframe"
bdims[,1]
bdims[,"height"]
bdims$height
#Select the second column in the dataframe
bdims[,2]
bdims[,"weight"]
bdims$weight
#Select the first row
bdims[1,]
#Select the last row
bdims[15,]
bdims[nrow(bdims),]
#Note: that bdims[i,j] selects the ith row and the jth column
bdims[10,2]
```

(c)

Summarize the data using the **summary()** function; get the mean and standard deviations for eachcolumn using the **apply()** and **sapply()** functions. What do the **apply()** and **sapply()** functions do?

```r
#Solution
#summarize the data
summary(bdims)
#get the mean of the data
apply(bdims,2,mean) #2 applies the function on columns.
sapply(bdims,mean)
lapply(bdims, mean) #Results will be a list
#the standard deviation
apply(bdims,2,sd)
sapply(bdims,sd)
lapply(bdims, sd)
```

(d)

Run a regression of height on weight using the lm() function, compare the results to those that use white standard errors using the coeftest() function, and plot the data.

```r
#Solution
#Run a linear regression of height on weight
bdims.model <- lm(height ~ weight,data=bdims)
#Look at a summary of the output
summary(bdims.model)
##White standard errors

# Here we need to install the sandwich package for the white standard errors
# install. packages("sandwich"), if you have it installed just load the package with library(sandwich)

require(lmtest);require(sandwich)


coeftest(bdims.model,vcov=sandwich)
# vcov = sandwich means that the variance / covariance matrix uses White
# standard errors (robustness checks)
#plot the data
plot(bdims)
pairs(bdims)
```

2.3

Write your first function in R, that is able to calculate the BMI

$$BMI = \frac{kg}{(\frac{cm}{100})^2}$$

Now add a column to the **bdims** dataframe with the BMI

```r
#Solution
#BMI function
bmi.func <- function(cm,kg) {
bmi <- kg/((cm/100)^2)
return(bmi)
}
```

```r
bdims$bmi <- bmi.func(bdims$height,bdims$weight)
bdims
```

2.4

In this exercise we will replicate the output of the lm()-function. So you have to think back to the classes on OLS. Now clear the workspace (for a new question). Then set the seed using the set.seed().set n = 100 for the number of random draws that we'll use and generate two random draws from the normal distribution with n data points, call them x1 and x2. (set.seed() makes sure that you the same random draws everytime you run the code)

(a)

Regress x1 on x2 and view a summary of the regression output. Use **set.seed(123456)** to set random seed for the random number generator used in rnorm().

```r
#Solution
#clear the workspace
rm(list=ls())
#Set the seed
set.seed(123456)
#the number of observations
n <- 100
#Two random series from the normal distribution
x1 <- rnorm(n)
x2 <- rnorm(n)
#(a) the model
model <- lm(x1 ~ x2)
#The output we will try and replicate
summary(model)
```

(b)

Assign x1 to Y (dependent variable) and a constant and x2 to X (independent variables). Program the OLS regression estimator using matrix notation, get the residuals, and compare them to the output from R's built in functions

$$\hat{\beta}_1 = (X'X)^{-1} * X'Y$$

$$\hat{\epsilon} = Y - X\hat{\beta}$$

```r
#Solution
#The dependent variable
Y <- x1
#The matrix of independent variables. Notice how we include the constant
X <- matrix(c(rep(1,n),x2),nrow=n,ncol=2)
#The regression estimator
bhat <- inv(t(X)%*%X)%*%t(X)%*%Y # (X'X)^(-1)X'Y
bhat #same as output from R!
```

```r
#Get the residuals
residuals <- Y - X%*%bhat
cor(residuals,model$residuals) #equals 1 same as R!
```

(c)

Get the standard errors of the residuals, the var-cov matrix and the standard errors for the regression estimators and compare them to R's built in functions.

```r
#Solution
#Get the variance of the residuals --> note that we have two
#independent variables
resid.var <- as.numeric(t(residuals)%*%residuals)*(1/(n-2))
#Get the standard error of the residuals --> in regression analysis,
#this is just the square root of the variance of the residuals
se.resid <- sqrt(resid.var)
se.resid #Same as R!
#Get the variance-cov matrix
varCov <- inv(t(X)%*%X)*resid.var
varCov
#Compare this to the var-cov matrix from R
vcov(model)
#They are the same!
#Get the standard errors using our varCov matrix
se <- sqrt(diag(varCov))
se
#compare these to the standard errors from R
sqrt(diag(vcov(model))) #Same!!
#or
summary(model)
```

(d)

Get the t-statistics, p-values for the t-statistics, F-statistic, and the p-value for the F-statistic.

```r
#Solution
#Now get the t-stats. Note: our null is that the regression estimators
#are equal to zero. This will be a two-sided test
tstat <- bhat/se
tstat
#Compare these to R
summary(model) #Same!
#p-values. this is a two sided test
#First get the degrees of freedom. df = n-2 since
#we one independent variable plus a constant
df <- n-2
#Use the cdf of the t distribution
pvals <- 2*(1 - pt(abs(tstat),df))
pvals #same as R!
#The F-statistic
#RSS under the null
RSS_0 <- as.numeric(t((Y-mean(Y)))%*%(Y-mean(Y)))
```

```r
#RSS under the alternative
RSS_1 <- as.numeric(t(residuals)%*%residuals)
#The numerator of the F-stat
Fnum <- (RSS_0 - RSS_1)/(2-1)
#The denominator of the F-stat
Fdenom <- (RSS_1)/(n-2)
#The F-stat
F <- Fnum/Fdenom
F #Same as R!
#The p-value for the f-stat
dfnum <- 2-1 #the degrees of freedom for the numerator
dfdenom <- n-2 #the df for the denominator
Fpval <- 1 - pf(F,dfnum,dfdenom)
Fpval #Same as R
```

(e)

Get the R2 and AdjR2 statistics.

```r
#Solution
#The R^2
#We already defined the Residual sum of squares above as RSS_1
#Define the total sum of squares
TSS <- as.numeric(t((Y-mean(Y)))%*%(Y-mean(Y)))
R2 <- 1 - RSS_1/TSS
R2 #same as R!
#The adjusted R^2 -- For the adjusted r-squared we
#divide the numerator and the denominator by the degrees
#of freedom.
R2_adj <- 1 - ((1/(n-2))*RSS_1)/((1/(n-1))*TSS)
R2_adj # Same as R!
```

Exercises 2 and 5 from fpp3 chapter 2

2

Use filter() to find what days corresponded to the peak closing price for each of the four stocks in gafa_stock.

```r
# Load the fpp3 package that corresponds to the book
library(fpp3)
#Solution
summary(gafa_stock)

gafa_stock %>%
  group_by(Symbol) %>%
  filter(Close == max(Close)) %>%
  ungroup() %>%
  select(Symbol, Date, Close)
```

5

(a)

Download tourism.xlsx from the book website and read it into R using read_excel() from the readxl package, http://OTexts.com/fpp3/extrafiles/tourism.xlsx.

```
#Solution
download.file("http://OTexts.com/fpp3/extrafiles/tourism.xlsx",
              tourism_file <- tempfile())
```

(b)

Create a tsibble which is identical to the tourism tsibble from the tsibble package.

```
#Solution
my_tourism <- readxl::read_excel(tourism_file) %>%
  mutate(Quarter = yearquarter(Quarter)) %>%
  as_tsibble(
    index = Quarter,
    key = c(Region, State, Purpose)
  )

my_tourism
```

(c)

Find what combination of Region and Purpose had the maximum number of overnight trips on average.

```
#Solution
my_tourism %>%
  as_tibble() %>%
  group_by(Region, Purpose) %>%
  summarise(Trips = mean(Trips)) %>%
  ungroup() %>%
  filter(Trips == max(Trips))
```

(d)

Create a new tsibble which combines the Purposes and Regions, and just has total trips by State.

```
#Solution
state_tourism <- my_tourism %>%
  group_by(State) %>%
  summarise(Trips = sum(Trips)) %>%
  ungroup()
state_tourism
```